EXPERIMENT – 2

AIM-To convert the ER diagram of the Indian E-commerce platform into a relational schema and implement the schema using MySQL with primary keys, foreign keys, NOT NULL, UNIQUE, ON DELETE CASCADE and ON DELETE SET NULL constraints, followed by insertion of sample data and demonstration of referential integrity violations.

OBJECTIVES-

- To convert an ER model into relational tables.
- To define primary keys.
- To define composite primary keys.
- To define foreign keys.
- To apply NOT NULL constraints.
- To apply UNIQUE constraints.
- To implement ON DELETE CASCADE.
- To implement ON DELETE SET NULL.
- To insert sample records.
- To demonstrate referential integrity violations.

THEORY-

1.Relational Schema

A relational schema represents the structure of a database in the form of relations (tables). Each relation consists of attributes (columns) and tuples (rows). The ER model is converted into a set of related tables so that the database can be implemented in a relational database management system.

2. Foreign Key

A foreign key is an attribute or group of attributes in one table that refers to the primary key of another table. It establishes a relationship between two tables and maintains referential integrity between them.

3. Referential Integrity

Referential integrity is a database constraint that ensures that a foreign-key value must correspond to an existing primary-key value in the referenced table, unless the foreign key permits NULL. It prevents invalid relationships between records in related tables.

4. NOT NULL Constraint

The NOT NULL constraint ensures that a column cannot contain a NULL value. It is used for attributes whose values are mandatory and must be provided when a record is inserted.

5. UNIQUE Constraint

The UNIQUE constraint ensures that all non-NULL values in a column or combination of columns are different. It prevents duplicate values from being stored in the specified attribute.

6. Entity-to-Relation Mapping

Entity-to-relation mapping is the process of converting entities from an ER diagram into relational tables. The attributes of an entity become columns, and the primary key of the entity becomes the primary key of the corresponding relation.

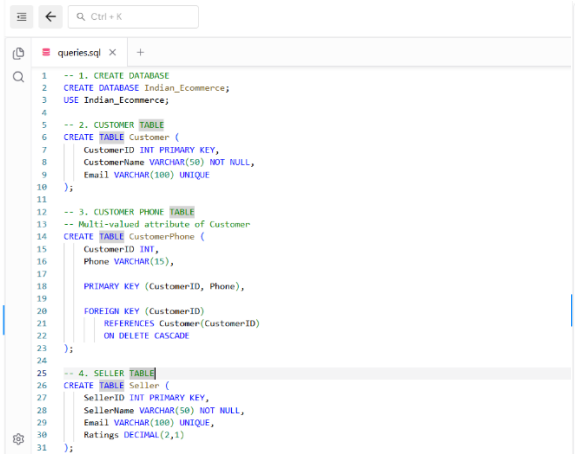
7. Relationship Mapping

Relationships in an ER diagram are represented in the relational model using foreign keys and, when necessary, separate relationship tables. The method of mapping depends on the cardinality and participation of the relationship.

8. Weak Entity Mapping

A weak entity is converted into a separate relation containing its own attributes and the primary key of its owner entity. The owner's primary key becomes part of the weak entity's composite primary key and also acts as a foreign key.

SOURCE CODE-



```sql
-- 1. CREATE DATABASE
CREATE DATABASE Indian_Ecommerce;
USE Indian_Ecommerce;

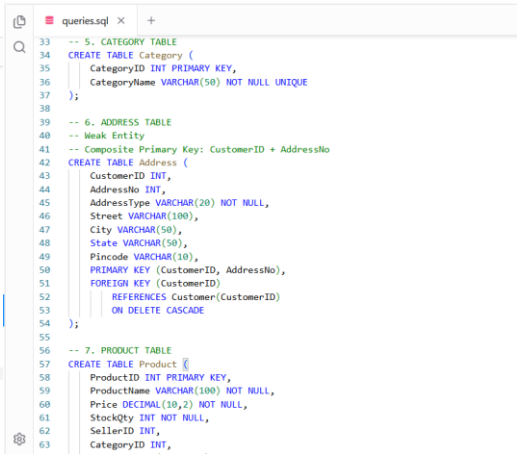
-- 2. CUSTOMER TABLE
CREATE TABLE Customer (
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE
);

-- 3. CUSTOMER PHONE TABLE
-- Multi-valued attribute of Customer
CREATE TABLE CustomerPhone (
    CustomerID INT,
    Phone VARCHAR(15),

    PRIMARY KEY (CustomerID, Phone),

    FOREIGN KEY (CustomerID)
        REFERENCES Customer(CustomerID)
        ON DELETE CASCADE
);

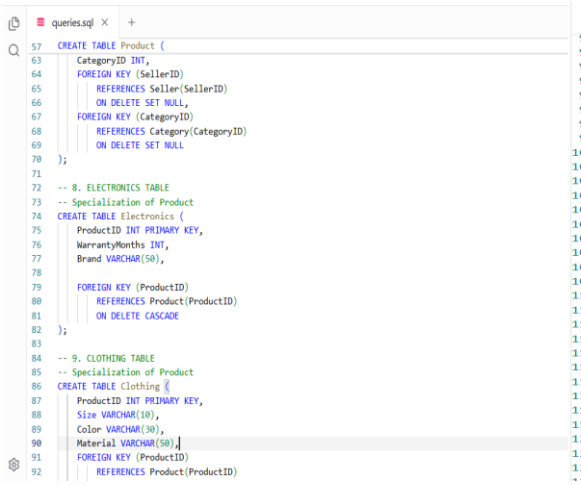
-- 4. SELLER TABLE
CREATE TABLE Seller (
    SellerID INT PRIMARY KEY,
    SellerName VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Ratings DECIMAL(2,1)
);
```



```sql
-- 5. CATEGORY TABLE
CREATE TABLE Category (
    CategoryID INT PRIMARY KEY,
    CategoryName VARCHAR(50) NOT NULL UNIQUE
);

-- 6. ADDRESS TABLE
-- Weak Entity
-- Composite Primary Key: CustomerID + AddressNo
CREATE TABLE Address (
    CustomerID INT,
    AddressNo INT,
    AddressType VARCHAR(20) NOT NULL,
    Street VARCHAR(100),
    City VARCHAR(50),
    State VARCHAR(50),
    Pincode VARCHAR(10),
    PRIMARY KEY (CustomerID, AddressNo),
    FOREIGN KEY (CustomerID)
        REFERENCES Customer(CustomerID)
        ON DELETE CASCADE
);

-- 7. PRODUCT TABLE
CREATE TABLE Product (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(100) NOT NULL,
    Price DECIMAL(10,2) NOT NULL,
    StockQty INT NOT NULL,
    SellerID INT,
    CategoryID INT,
```

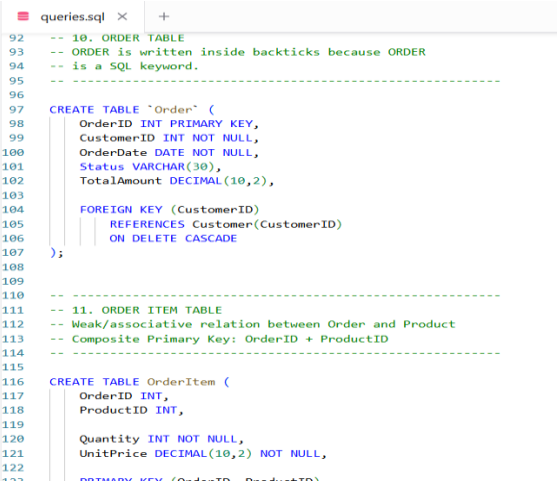


```sql
CREATE TABLE Product (
    CategoryID INT,
    FOREIGN KEY (SellerID)
        REFERENCES Seller(SellerID)
        ON DELETE SET NULL,
    FOREIGN KEY (CategoryID)
        REFERENCES Category(CategoryID)
        ON DELETE SET NULL
);

-- 8. ELECTRONICS TABLE
-- Specialization of Product
CREATE TABLE Electronics (
    ProductID INT PRIMARY KEY,
    WarrantyMonths INT,
    Brand VARCHAR(50),

    FOREIGN KEY (ProductID)
        REFERENCES Product(ProductID)
        ON DELETE CASCADE
);

-- 9. CLOTHING TABLE
-- Specialization of Product
CREATE TABLE Clothing (
    ProductID INT PRIMARY KEY,
    Size VARCHAR(10),
    Color VARCHAR(30),
    Material VARCHAR(50),
    FOREIGN KEY (ProductID)
        REFERENCES Product(ProductID)
```



```sql
-- 10. ORDER TABLE
-- ORDER is written inside backticks because ORDER
-- is a SQL keyword.
-- ------------------------------------------------------------

CREATE TABLE `Order` (
    OrderID INT PRIMARY KEY,
    CustomerID INT NOT NULL,
    OrderDate DATE NOT NULL,
    Status VARCHAR(30),
    TotalAmount DECIMAL(10,2),

    FOREIGN KEY (CustomerID)
        REFERENCES Customer(CustomerID)
        ON DELETE CASCADE
);


-- ------------------------------------------------------------
-- 11. ORDER ITEM TABLE
-- Weak/associative relation between Order and Product
-- Composite Primary Key: OrderID + ProductID
-- ------------------------------------------------------------

CREATE TABLE OrderItem (
    OrderID INT,
    ProductID INT,

    Quantity INT NOT NULL,
    UnitPrice DECIMAL(10,2) NOT NULL,
```

```sql
    PRIMARY KEY (OrderID, ProductID),

    FOREIGN KEY (OrderID)
        REFERENCES `Order`(OrderID)
        ON DELETE CASCADE,

    FOREIGN KEY (ProductID)
        REFERENCES Product(ProductID)
        ON DELETE CASCADE
);


-- ------------------------------------------------
-- 12. PAYMENT TABLE
-- One Payment for one Order
-- ------------------------------------------------

CREATE TABLE Payment (
    PaymentID INT PRIMARY KEY,
    OrderID INT NOT NULL UNIQUE,
    PaymentDate DATE,
    Amount DECIMAL(10,2),
    PaymentMethod VARCHAR(30),
    Status VARCHAR(30),

    FOREIGN KEY (OrderID)
        REFERENCES `Order`(OrderID)
        ON DELETE CASCADE
);
```

queries.sql

```sql
-- ------------------------------------------------
-- 13. DELIVERY TABLE
-- One Delivery for one Order
-- ------------------------------------------------

CREATE TABLE Delivery (
    DeliveryID INT PRIMARY KEY,
    OrderID INT NOT NULL UNIQUE,
    DeliveryDate DATE,
    DeliveryStatus VARCHAR(30),
    TrackingNo VARCHAR(50) UNIQUE,

    FOREIGN KEY (OrderID)
        REFERENCES `Order`(OrderID)
        ON DELETE CASCADE
);
-- 14. INSERT CUSTOMERS
-- ------------------------------------------------

INSERT INTO Customer
(CustomerID, CustomerName, Email)
VALUES
(101, 'Riddhima', 'riddhima@gmail.com'),
(102, 'Aarav', 'aarav@gmail.com'),
(103, 'Ananya', 'ananya@gmail.com'),
(104, 'Rahul', 'rahul@gmail.com'),
(105, 'Priya', 'priya@gmail.com');
```

queries.sql MySQL

```sql
-- ------------------------------------------------
-- 15. INSERT CUSTOMER PHONE NUMBERS
-- ------------------------------------------------

INSERT INTO CustomerPhone
(CustomerID, Phone)
VALUES
(101, '9876543210'),
(101, '9123456780'),
(102, '9876501234'),
(103, '9988776655'),
(104, '9090909090'),
(105, '8888888888');


-- ------------------------------------------------
-- 16. INSERT SELLERS
-- ------------------------------------------------

INSERT INTO Seller
(SellerID, SellerName, Email, Ratings)
VALUES
(201, 'TechWorld', 'techworld@gmail.com', 4.5),
(202, 'FashionHub', 'fashionhub@gmail.com', 4.2),
(203, 'HomeStore', 'homestore@gmail.com', 4.0),
(204, 'BookMart', 'bookmart@gmail.com', 4.3),
(205, 'SmartShop', 'smartshop@gmail.com', 4.6);


-- ------------------------------------------------
-- 17. INSERT CATEGORIES
```

queries.sql MySQL

```sql
INSERT INTO Category
(CategoryID, CategoryName)
VALUES
(301, 'Electronics'),
(302, 'Clothing'),
(303, 'Books'),
(304, 'Home Appliances'),
(305, 'Accessories');

-- 18. INSERT ADDRESSES
-- ------------------------------------------------

INSERT INTO Address
(CustomerID, AddressNo, AddressType, Street, City, State, Pincode)
VALUES
(101, 1, 'Home', 'MG Road', 'Delhi', 'Delhi', '110001'),
(101, 2, 'Office', 'Sector 18', 'Noida', 'Uttar Pradesh', '201301'),
(102, 1, 'Home', 'Park Street', 'Kolkata', 'West Bengal', '700016'),
(103, 1, 'Home', 'FC Road', 'Pune', 'Maharashtra', '411004'),
(104, 1, 'Home', 'Banjara Hills', 'Hyderabad', 'Telangana', '500034'),
(105, 1, 'Home', 'MG Road', 'Bengaluru', 'Karnataka', '560001');


-- ------------------------------------------------
-- 19. INSERT PRODUCTS
-- ------------------------------------------------

INSERT INTO Product
(ProductID, ProductName, Price, StockQty, SellerID, CategoryID)
VALUES
(401, 'Laptop', 65000.00, 10, 201, 301),
```

queries.sql

```sql
(402, 'Smartphone', 30000.00, 20, 201, 301),
(403, 'T-Shirt', 999.00, 50, 202, 302),
(404, 'Jeans', 1999.00, 30, 202, 302),
(405, 'Mixer', 4500.00, 15, 203, 304);


-- ------------------------------------------------
-- 20. INSERT ELECTRONICS PRODUCTS
-- ------------------------------------------------

INSERT INTO Electronics
(ProductID, WarrantyMonths, Brand)
VALUES
(401, 24, 'Dell'),
(402, 12, 'Samsung'),
(405, 18, 'Philips');


-- ------------------------------------------------
-- 21. INSERT CLOTHING PRODUCTS
-- ------------------------------------------------

INSERT INTO Clothing
(ProductID, Size, Color, Material)
VALUES
(403, 'M', 'Black', 'Cotton'),
(404, '32', 'Blue', 'Denim');


-- ------------------------------------------------
-- 22. INSERT ORDERS
```

queries.sql

```sql
INSERT INTO `Order`
(OrderID, CustomerID, OrderDate, Status, TotalAmount)
VALUES
(501, 101, '2026-10-01', 'Delivered', 65000.00),
(502, 102, '2026-10-02', 'Shipped', 30000.00),
(503, 103, '2026-10-03', 'Processing', 999.00),
(504, 104, '2026-10-04', 'Delivered', 1999.00),
(505, 105, '2026-10-05', 'Processing', 4500.00);


-- ------------------------------------------------
-- 23. INSERT ORDER ITEMS
-- ------------------------------------------------

INSERT INTO OrderItem
(OrderID, ProductID, Quantity, UnitPrice)
VALUES
(501, 401, 1, 65000.00),
(502, 402, 1, 30000.00),
(503, 403, 1, 999.00),
(504, 404, 1, 1999.00),
(505, 405, 1, 4500.00);

-- 24. INSERT PAYMENTS
-- ------------------------------------------------

INSERT INTO Payment
(PaymentID, OrderID, PaymentDate, Amount, PaymentMethod, Status)
VALUES
(601, 501, '2026-10-01', 65000.00, 'UPI', 'Successful'),
```

queries.sql

```sql
(603, 503, '2026-10-03', 999.00, 'UPI', 'Successful'),
(604, 504, '2026-10-04', 1999.00, 'Net Banking', 'Successful'),
(605, 505, '2026-10-05', 4500.00, 'Cash', 'Pending');


-- ------------------------------------------------------
-- 25. INSERT DELIVERY DETAILS
-- ------------------------------------------------------

INSERT INTO Delivery
(DeliveryID, OrderID, DeliveryDate, DeliveryStatus, TrackingNo)
VALUES
(701, 501, '2026-10-04', 'Delivered', 'TRK1001'),
(702, 502, '2026-10-06', 'Shipped', 'TRK1002'),
(703, 503, NULL, 'Processing', 'TRK1003'),
(704, 504, '2026-10-07', 'Delivered', 'TRK1004'),
(705, 505, NULL, 'Processing', 'TRK1005');


-- ======================================================
-- DISPLAY DATA
-- ======================================================


-- Display Customer table

SELECT * FROM Customer;


-- Display Customer Phone table

```

queries.sql

```sql
SELECT * FROM CustomerPhone;


-- Display Seller table

SELECT * FROM Seller;


-- Display Category table

SELECT * FROM Category;


-- Display Address table

SELECT * FROM Address;


-- Display Product table

SELECT * FROM Product;


-- Display Electronics table

SELECT * FROM Electronics;


-- Display Clothing table

SELECT * FROM Clothing;
```

queries.sql

```sql
-- Display Order table

SELECT * FROM `Order`;


-- Display OrderItem table

SELECT * FROM OrderItem;


-- Display Payment table

SELECT * FROM Payment;


-- Display Delivery table

SELECT * FROM Delivery;
-- REFERENTIAL INTEGRITY VIOLATIONS
-- ======================================================


-- ------------------------------------------------------
-- VIOLATION 1:
-- SellerID 999 does not exist in Seller table.
-- Therefore, this INSERT will fail.
-- ------------------------------------------------------

INSERT INTO Product
(ProductID, ProductName, Price, StockQty, SellerID, CategoryID)
```

queries.sql

```sql
(406, 'Tablet', 18000.00, 8, 999, 301);


-- ------------------------------------------------------
-- VIOLATION 2:
-- ProductID 999 does not exist in Product table.
-- Therefore, this INSERT will fail.
-- ------------------------------------------------------

INSERT INTO OrderItem
(OrderID, ProductID, Quantity, UnitPrice)
VALUES
(501, 999, 1, 1000.00);


-- ------------------------------------------------------
-- VIOLATION 3:
-- CustomerID 999 does not exist in Customer table.
-- Therefore, this INSERT will fail.
-- ------------------------------------------------------

INSERT INTO `Order`
(OrderID, CustomerID, OrderDate, Status, TotalAmount)
VALUES
(506, 999, '2026-10-06', 'Processing', 5000.00);


-- ======================================================
-- DEMONSTRATION OF ON DELETE CASCADE
-- ======================================================

```

queries.sql

```sql
-- Create a temporary customer for demonstration

INSERT INTO Customer
(CustomerID, CustomerName, Email)
VALUES
(106, 'TestUser', 'testuser@gmail.com');


INSERT INTO Address
(CustomerID, AddressNo, AddressType, Street, City, State, Pincode)
VALUES
(106, 1, 'Home', 'Test Street', 'Delhi', 'Delhi', '110002');


INSERT INTO CustomerPhone
(CustomerID, Phone)
VALUES
(106, '9000000000');

-- Check data before deleting customer

SELECT * FROM Customer
WHERE CustomerID = 106;

SELECT * FROM Address
WHERE CustomerID = 106;

SELECT * FROM CustomerPhone
WHERE CustomerID = 106;
```

queries.sql

```sql
-- Delete customer

DELETE FROM Customer
WHERE CustomerID = 106;


-- Because ON DELETE CASCADE is used,
-- related Address and CustomerPhone records
-- are automatically deleted.


SELECT * FROM Address
WHERE CustomerID = 106;

SELECT * FROM CustomerPhone
WHERE CustomerID = 106;


-- ======================================================
-- DEMONSTRATION OF ON DELETE SET NULL
-- ======================================================


-- Check product before deleting seller

SELECT * FROM Product
WHERE SellerID = 205;


-- Delete seller 205
```

queries.sql

```sql
DELETE FROM Seller
WHERE SellerID = 205;


-- Any Product referring to SellerID 205
-- will have SellerID changed to NULL.


-- ============================================================
-- FINAL CHECK
-- ============================================================

SELECT * FROM Customer;

SELECT * FROM Seller;

SELECT * FROM Category;

SELECT * FROM Product;

SELECT * FROM Electronics;

SELECT * FROM Clothing;

SELECT * FROM `Order`;

SELECT * FROM OrderItem;

SELECT * FROM Payment;

SELECT * FROM Delivery;
```

OUTPUT-

1. The ER diagram was successfully converted into relational tables using MySQL.
2. Primary keys, foreign keys, NOT NULL, UNIQUE, ON DELETE CASCADE, and ON DELETE SET NULL constraints were implemented.
3. Sample data was successfully inserted into all the tables.
4. Referential integrity violations were successfully demonstrated using invalid foreign-key values.
5. Cascade and Set NULL operations were successfully verified.

OBSERVATION/OUTCOME-

1. The ER model was successfully mapped into a relational database schema.
2. Primary and foreign keys maintained relationships between the tables.
3. Constraints helped maintain data accuracy and integrity.
4. Invalid foreign-key values were rejected by the database.
5. Thus, the Indian E-commerce database was successfully implemented in MySQL.